UNIVERSIDAD DE GUADALAJARA
CENTRO UNIVERSITARIO DE LOS VALLES

Anti-Corruption Base Intelligence, AcBi

Prepared by:

Hiram Gabriel Alonso Marquez

Index

Project Charter.....3
Development Methodology.....4
Analyze requirements.....5
Software Requirements Specification (SRS) for AcBi.....5
Product Perspective.....5
User Classes and Characteristics.....6
Specific Requirements.....6
Functional Requirements.....7
Non-Functional Requirements.....9
Contract Type.....10
Design Overview.....12
Resource Planning.....13
Material Resources.....15
Item Configuration.....15
Change Request.....15

Project Charter

The “AcBi” project consists of designing and developing a cross-platform mobile application (Android and iOS) aimed at researchers, journalists, and civil society organizations to proactively detect corruption patterns in the public sector. The solution is based on a local-first, offline approach, integrating five main functional modules: Data Listener (real-time capture and monitoring of public data), Network Analysis (construction of dynamic relationship graphs and risk scoring), Process Tracking (detailed monitoring of tenders and bureaucratic processes), Relational Time Lines (RTL, temporal comparison of declarations vs. captured signals), and Time Paradox (central dashboard with alerts and detection of chronological inconsistencies).

The main objective is to provide a forensic intelligence tool that allows for the identification of anomalies such as irregular awards, networks of hidden beneficiaries, and “time paradoxes” (approvals before official requests).

The platform incorporates 20 non-functional extreme security requirements, including end-to-end encryption (E2EE), an immutable digital chain of custody using SHA-512 hashes and certified timestamps, operational anonymity through native Tor/VPN integration and automatic IP rotation, resistance to counterintelligence (sandbox detection, memory dump prevention, reverse honeypot, and programmable self-destruct), as well as protections against evidence tampering and reverse analysis.

The application is built on a Clean Architecture, with a local-first design, using Tauri 2.0 Mobile and a Rust backend for obfuscation. A key innovation is the interactive API Manager, which allows the user (detective or investigator) to manually configure data sources provided for the case (URLs, endpoints, parsing rules), visualize them in a dynamic diagram (with nodes and connections), and validate them in real time.

Limitations of the Data of Apis (LDA) for version 001:

- Everything requires manual API configuration
- New companies, changes in the Public Registry of Commerce, partners, and legal representatives.
- Opening bank accounts, financial transactions of family members or relatives, and inter-institutional transfers.
- Updates to properties, houses, land, cars, motorcycles registered in the name of family members or close associates.
- Locations, from apps or spyware.
- Public works announcements, deadlines, awards, bids submitted, winners, budget transfers, and dates.

- Data from public projects.
- Contact information.
- Official declarative data: asset declarations, declarations of interests, public statements of officials.

Data that it does NOT support:

- Chats or personal messages
- Confidential bank details
- Any restricted data not included in the LDA

Development Methodology

The "AcBi" project is a multiplatform mobile application designed for researchers for the proactive detection of corruption. We will utilize an Agile methodology (Scrum). The system follows a Clean Architecture approach, organizing code into independent layers to facilitate maintenance:

- Development Approach It will follow a Hybrid Development model: React with JavaScript for interactive UIs like dashboards, timelines, and charts via vis.js and D3.js with Rust for logic bussiness and obfuscation.

- The system uses Tauri with React Native for a multiplatform app, PC, and Android.

- The structure is Clean Architecture with Local-First (Storage). To maintain total control over sensitive data, the app runs without third-party central servers.

- FrontendUI is made by React Native with vis.js and D3.js for the dashboard, visualizations, and dynamic diagrams.

- Backend will be made with Rust for main modules, API management using Go to make many requests at the same time.

- The Encrypted Database is local storage using SQLCipher in the test devices.

- Comprehensive testing will be conducted for unit tests for cryptography, in order to ensure anonymity and counter-intelligence protection.

- SHA family for hashing, cryptography, VPN support, and custom Rust-based anti-forensics (Obfuscator.io for code protection), sandbox, and memory dump detection to prevent the malicious agent from obtaining a copy of the contents of RAM at a specific time.

Analyze requirements

Software Requirements Specification (SRS) for AcBi

The purpose of this Software Requirements Specification (SRS) is to define the functional and non-functional requirements for "Paradox Mobile," a cross-platform mobile application designed to assist investigators.

Definitions, Acronyms, and Abbreviations

API: Application Programming Interface – Interfaces for data exchange with external public sources.

1. E2EE: End-to-End Encryption – Encryption ensuring data is secure from source to destination.
2. RTL: Relational Time Lines – Chronological views comparing official and captured data.
3. SHA: Secure Hash Algorithm – A cryptographic hash function for data integrity.
4. VPN: Virtual Private Network – A secure internet conection traffic.
5. Noise Threshold: A configurable metric for detecting inconsistencies in data timelines.
6. Honeypot Inverso: A defensive mechanism generating fake data upon intrusion detection.
7. Local-First: Design where all processing occurs on the device.

Product Perspective

"It is a novel tool in the domain of anti-corruption technology."

The application enables users to:

- Capture and monitor public data changes (e.g., new companies, financial movements, geolocations) via configurable APIs.
- Build and visualize dynamic graphs of relationships between officials, families, and entities, with risk scoring.
- Track full lifecycles of bureaucratic processes, flagging anomalies like ignored bids or accelerated timelines.
- Generate relational timelines comparing official declarations to captured signals, calculating corruption probabilities.

User Classes and Characteristics

Primary Users: Investigators and detectives – Experienced analysts requiring forensic tools; Assume technical proficiency in data interpretation but not in programming.

Specific Requirements

Functions Requirements are organized by module:

1. Module 1: Data Listener (REQ-1.1 to REQ-1.10): Integrate/verify APIs; capture metadata/changes in registries, finances, properties, geolocations; monitor projects/funds; ensure API resilience.
2. Module 2: Network Analysis (REQ-2.1 to REQ-2.10): Build/update graphs; cross-reference identifiers; score risks; detect transfers/groups; visualize/filter data.
3. Module 3: Process Tracking (REQ-3.1 to REQ-3.10): Trace lifecycles; flag irregularities; compare offers; cross with networks; visualize timelines.
4. Module 4: RTL (REQ-4.1 to REQ-4.10): Generate/compare timelines; calculate noise/paradoxes; detect contradictions; export as PDF.
5. Module 5: Time Paradox (REQ-5.1 to REQ-5.10): Dashboard with filters/alerts; flag cases; interactive maps/timelines; export reports.
6. API Manager (REQ-6.1 to REQ-6.5): Manual CRUD for APIs; dynamic parsing; validation; interactive diagram visualization.

Functional Requirements

50 Functionalities:

1. Module 1: Data Listener
    a. API Integration
    b. Verification of Manipulated APIs
    c. Metadata Capture
    d. Listening for changes.
    e. Detection of bank account openings or public financial transactions of family members.
    f. Monitoring of changes in properties and vehicles registered in the names of family members or acquaintances.
    g. API Editing: DELETE, EDIT
    h. Manipulated API Report
    i. OSINT INTEGRATION
    j. API MANAGER.

2. Module 2: Network Analysis
    a. Construction of dynamic relationship graphs (officials ↔ family members ↔ companies).
    b. Graph filtering by politician and time period.
    c. Massive data cross-referencing.
    d. Cross-referencing of projects with information from Module 1.
    e. Risk scoring by degree of closeness.
    f. Detection of "shared accounts" or transfers between close circles.
    g. Mapping of closed groups or chats where officials and beneficiaries appear in lists.
    h. Interactive visualization of graphs with zoom.
    i. Graph update button.
    j. Filters

3. Module 3: Process Tracking
    a. Tracing the lifecycle of the data collected for the case by day, in PDF table format.
    b. Tracing by name.
    c. Monitoring of unusual accelerations in bureaucratic deadlines.
    d. Historical record of each data collection by package.
    e. AUTOMATIC COMPARISON BETWEEN SUBMITTED OFFERS AND THE WINNING ONE, BY FILE NAME
    f. Tracing of beneficiaries identified in the network graph.
    g. Tracing of budget transfers between departments
    h. Tracing of approvals.

    i. Preparation of information in timelines that encompass Module 3
    j. Visual timeline by process with highlighted anomalies.

4. Module 4: Relational Timelines (RTL)
    a. Generation of relational timelines by politician/institution/project.
    b. Calculation of the "noise threshold" (inconsistencies between declared and observed information).
    c. Temporal comparison: official statements vs. captured signals.
    d. Detection of approvals before official requests (Basic Time Paradox).
    e. Algorithms to predict the probability of corruption based on historical noise (Time Paradox).
    f. User-configurable threshold (e.g., >25% noise = high alert).
    g. Comparative visualization of timelines (official vs. observed).
    h. Detection of unexplained "skips" in the authorization flow. Recording of specific contradictions (e.g., "politician claims to be unaware of the company, but a family member is a partner").
    i. Export of RTLs as PDFs

5. Module 5: Time Paradox
    a. Main dashboard with filters by date, politician, institution, and project.
    b. Chronological display of all RTLs with noise thresholds.
    c. Push/email/SMS alerts when the noise threshold is exceeded.
    d. Automatic flag: "Start Investigation Case" with collected evidence.
    e. Interactive risk map by state/municipality.
    f. Bulk export of reports for NGOs, journalists, or authorities.
    g. "Paradox" mode: highlights serious temporal inconsistencies in red.
    h. History of detected cases.
    i. Interactive interface with animations.
    j. Timeline with information sheets.

Non-Functional Requirements

Non-Functional Requirements:

1. Data Security: All collected and processed data must be encrypted at rest and in transit.
2. Forensic-Level Security: End-to-end encryption (E2EE).
3. Immutable Digital Chain of Custody: Every piece of data captured, analyzed, or exported must have a certified SHA-512 hash and timestamp.
4. Operational Anonymity of the Investigator: Native support for VPN/Tor + automatic IP rotation.
5. Counterintelligence Resistance: Automatic detection and blocking of tracking attempts.
6. Offline Availability.
7. Scalability for Large Cases.
8. Tolerance for Outages.
9. Professional and Technical Interface: Dashboard designed for analysts.
10. Protection Against Evidence Tampering: Automatic detection of attempts to edit or inject false data.
11. Sandbox Detection and Reverse Analysis: The system must self-destruct or enter "false positive" mode if it detects that it is being analyzed in a virtual or debug environment. Code obfuscation: Binary code and scripts obfuscated with multiple layers (VMProtect, Themida, or equivalent) to hinder reverse engineering.
12. Memory dumping protection: Anti-memory dump mechanisms (anti-Process Explorer, anti-Cheat Engine, anti-DumpIt).
13. Automatic key and certificate rotation: Daily generation and renewal of encryption keys and certificates.
14. Keylogger and screen recorder detection: Automatic blocking or masking of sensitive input if capture tools are detected.
15. Traffic analysis protection: Use of padding, timing randomization, and dummy traffic to hide communication patterns.
16. Programmable self-destruction.
17. Reverse honeypot mode: The system can generate false data or deceptive signals if it detects intrusion attempts.
18. Internal researcher audit.
19. API monitoring and encryption.

Contract Type

CONFIDENTIALITY AND NON-DISCLOSURE AGREEMENT
This Agreement is entered into by and between:

[FULL NAME OF THE PUBLIC AGENCY OR ENTITY], represented herein by [NAME OF THE AUTHORIZED OFFICIAL], in their capacity as [Job Title], hereinafter referred to as "THE ENTITY" or the "DISCLOSING PARTY";

And, on the other part:

[FULL NAME OR CORPORATE NAME OF THE CONTRACTOR / COMPANY / INDIVIDUAL], represented herein by [NAME OF THE LEGAL REPRESENTATIVE], with its registered address at [Full Address], hereinafter referred to as "THE CONTRACTOR" or the "RECIPIENT PARTY";

Both parties mutually recognize each other's legal capacity to enter into this instrument and, under penalty of perjury, state the following:

REPRESENTATIONS
I. THE ENTITY is an agency / body / state-owned entity of the Government of the
United Mexican States / the Government of the State of [•], with the powers conferred upon it by
[mention law or decree of creation, e.g., Organic Law of the Federal Public Administration, etc.].

II. THE CONTRACTOR has been selected / awarded / hired to provide services / develop the project / collaborate in [brief description of the main contract object, e.g., "AcBi Development," "Cybersecurity Consulting Services," etc.], pursuant to contract / service order / agreement number [•] dated [DD/MM/YYYY] (hereinafter the "Main Contract").

III. For the execution of the Main Contract, THE ENTITY will provide or allow access to certain information, documents,
data, systems, databases, technical specifications, source codes, plans, strategies, reserved, classified,
or sensitive information (hereinafter the "Confidential Information").

IV. Both parties have an interest in ensuring that such information is adequately protected and not disclosed, reproduced, used, or exploited outside of the purposes strictly necessary to fulfill the Main Contract.

V. The parties acknowledge that the Confidential Information may constitute industrial secrets, reserved information, confidential information, or personal data held by obligated subjects, as applicable, under the terms of the governing legislation.

Therefore, the parties agree to enter into this CONFIDENTIALITY AGREEMENT subject to the following:

CLAUSES
FIRST. DEFINITION OF CONFIDENTIAL INFORMATION Confidential Information shall mean all information in any format
(written, electronic, oral, visual, magnetic, digital, etc.) that THE ENTITY provides or to which THE CONTRACTOR has access by
virtue of the Main Contract, including but not limited to:

Personal data.

Reserved or classified information.

Technical specifications, plans, diagrams, algorithms, source codes, and objects.

Strategies, methodologies, know-how, and processes.

Financial, operational, security, or intelligence information.

Any data that THE ENTITY marks as confidential or which, by its nature, should be treated as such.

SECOND. OBLIGATIONS OF THE CONTRACTOR THE CONTRACTOR agrees to:
a) Use the Confidential Information exclusively to fulfill the object of the Main Contract.
b) Not disclose, reproduce, copy, transmit, publish, or communicate the Confidential Information to third parties by any means without the express written authorization of THE ENTITY.
c) Implement appropriate physical, technical,
and administrative security measures to protect the Confidential Information (at least at the same level as it protects its own sensitive information). d) Limit access to the Confidential Information only to personnel for whom it is strictly necessary and who have signed an individual confidentiality commitment with equivalent obligations.
e) Immediately notify THE ENTITY of any access, use, disclosure, or suspected breach of confidentiality.
f) Return or destroy (and certify the destruction of) all Confidential Information upon termination of the Main Contract or when requested by THE ENTITY.

THIRD. EXCEPTIONS Information shall not be considered Confidential Information if it:
a) Is or becomes part of the public domain through no breach of this Agreement.
b) Was already legitimately known by THE CONTRACTOR prior to receipt.
c) Is received in good faith from a third party without an obligation of confidentiality.
d) Must be disclosed by legal mandate, final judicial resolution, or requirement of a competent authority (in which case THE ENTITY must be notified in advance so it may exercise its rights of defense).

FOURTH. TERM (VALIDITY) This Agreement shall enter into force upon its signing and shall remain in effect during the execution of the Main Contract and for an additional term of [10 years] from the date of its termination, or for as long as the information retains its reserved or confidential status according to the law.

FIFTH. LIABILITY FOR BREACH Any breach shall be considered a serious cause for termination of the Main Contract and shall give rise to:

Civil and/or criminal liability.

Payment of damages and losses.

Application of administrative sanctions provided for in the General Law of Administrative Responsibilities and other applicable public sector laws.

SIXTH. GOVERNING LAW AND JURISDICTION This Agreement shall be governed by the laws of the United Mexican States.
For all matters related to its interpretation and fulfillment, the parties submit to the jurisdiction of the competent courts located in [Mexico City / State Capital / Municipality], waiving any other jurisdiction.

Having read this document and being aware of its content and legal scope, the parties sign it in agreement in [City], on the [Day] of [Month], [Year].

FOR THE ENTITY [NAME AND TITLE] [Signature and Seal]

FOR THE CONTRACTOR [NAME OF LEGAL REPRESENTATIVE] [Signature and Seal]

WITNESSES (Optional) [Name and Signature] | [Name and Signature]

Design Overview

The diagram illustrates how AcBi's five core modules interact in a pipeline flow. M1: Data Listener acts as the entry point, ingesting raw data and feeding it simultaneously into M2: Network Analysis and M3: Process Tracking, which analyze relationships and monitor procedural activity independently. Both modules converge into M4: Relational Timelines, which cross-references findings to construct a chronological map of events and connections. All outputs then consolidate into M5: Time Paradox Dashboard, which synthesizes the full investigative picture and delivers it as visualizations and alerts to the end user — the detective. Table 1.1 Design Overview.

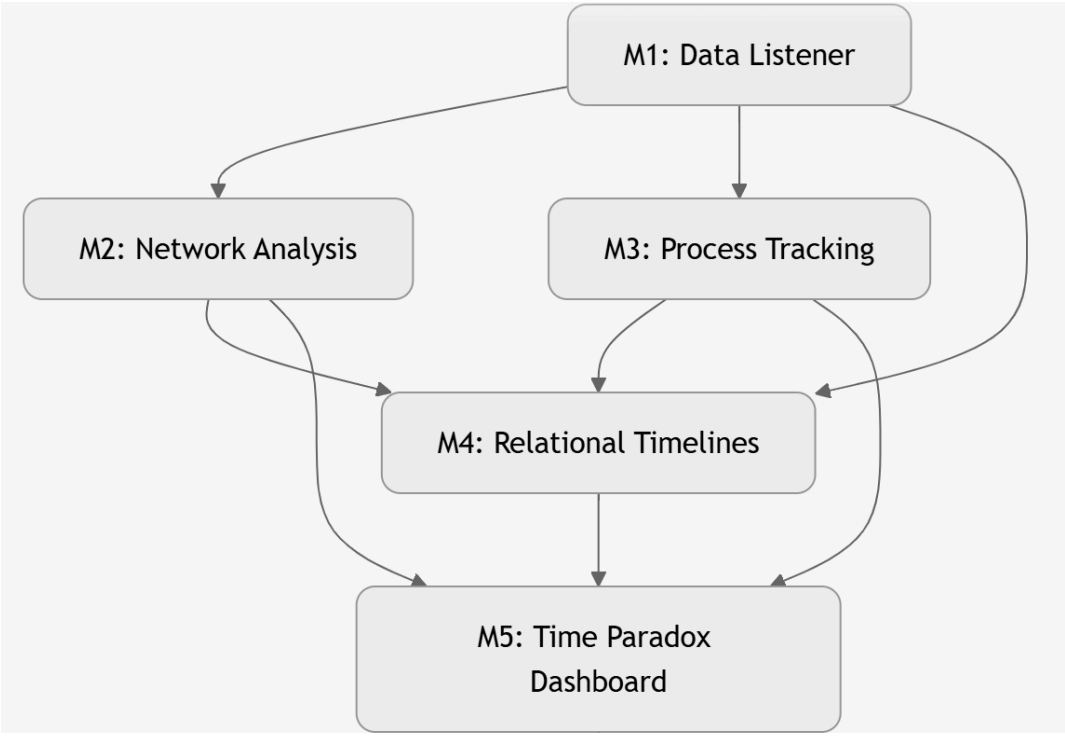

Table 1.1 Design Overview

Resource Planning

Resource planning for the "AcBi" project allocates human, material, and financial assets to achieve the development objectives for the mobile/PC application within a 6-month.

The following technologies were selected for the development of the "AcBi" project, as shown in Table 1 Technologies & Table 2 Team (8 people – 6 months full-time):

Category	Technology	Purpose/Goal
Frontend (UI & Visualizations)	React Native (with JavaScript)	Build the main dashboard, timelines, and interactive charts.
Frontend (UI & Visualizations)	vis.js & D3.js	Create dynamic diagrams.
Backend/Core (Business Logic)	Rust	Implement the obfuscated core, including entities (e.g., Officials, Companies) and main modules; protection against reverse engineering.
Backend/Core (Business Logic)	Go & Python	Simpler concurrent logic for modules
Framework & Multiplatform	Tauri	Bridge between frontend (React) and backend (Rust), handling IPC/Commands.
Database & Storage	SQLCipher	Encrypted local storage.
Database & Storage	Realm	Embedded mobile DB for fast queries in high-volume local data scenarios.
Security & Cryptography	SHA Family	Hash generation.
Security & Cryptography	VPN	VPN support.
Security & Cryptography	Custom Rust (Anti-forensics)	Sandbox and memory dump detection.
Security & Cryptography	Obfuscator.io (JS)	Code obfuscation for frontend/backend to prevent reverse engineering.

Infrastructure & Resilience	Local-first	Full offline operation; resilient API client via Tor for external public registries.
Testing & Quality	Jest (JS) & Cargo test (Rust)	Unit testing for frontend and backend code.

Table 1 Technologies

#	Role	Level	Qty	Responsibilities	Monthly Salary (MXN)	6-Month Total (MXN)
1	Sr. Crypto & Security	Senior	1	E2EE, anti-forensics, self-destruct, sandbox detection.	68k – 75k	~426,000
2	Sr. Rust & Architecture	Senior	1	Binary obfuscation, Tor integration, anti-dumping.	65k – 72k	~411,000
3	Frontend Developer	Mid	1	React, TS, D3.js, interactive dashboards.	32k – 42k	~222,000
4	Backend Rust Dev	Mid	1	Data Listener, Network Analysis, Process Tracking.	32k – 40k	~216,000
5	Integration / Mobile	Mid/Jr+	1	Tauri 2.0, IPC, Rust ↔ JS commands, bug fixing.	28k – 38k	~198,000
6	Full-Stack / Support	Jr/Mid	1	General support for FE/BE, cross-cutting tasks.	25k – 35k	~180,000
7	Tester / Security Eng.	Senior	1	Pentesting, OWASP Mobile, E2EE validation.	45k – 55k	~300,000

8	PM / Scrum Master	—	1	Coordination, backlog, agile ceremonies.	22k – 28k	~150,000

Table 2. Team (8 people – 6 months full-time)

Personnel Cost Summary
2 Senior Crypto/Architecture Developers: ≈ MXN 837,000
4 Mid/Junior Developers: ≈ MXN 816,000
1 Senior Security Tester: ≈ MXN 300,000
1Project Manager: ≈ MXN 150,000

Total Personnel Costs for 6 Months ≈ MXN 2,103,000 – 2,350,000

Total quoted amount: 12,000,000 MXN

Material Resources

Main Tools (Cost ≈ 0): VS Code, Rust, Tauri, React, GitHub, GitLab, ClickUp / Linear / Jira, Discord/Slack

Physical Testing Devices: 5 Mid-to-High-End Android/iOS Devices for Real-World Testing → MXN 80,000 – 110,000 (initial investment)

2 computers: MXN 100,000

Item Configuration

Change Request

Change Request Description: Incorporation of User-API History for Auditing
Request Date: February 18, 2026

Requester: Government

Impact on the Project:

Affected Modules: Module 1: Data Listener, RF-007: API EDITING, RF-010: API MANAGER

Planning and Timeline:

Adds 1 additional week of work, distributed across development, integration, and testing phases. This extends the total timeline from 6 months to approximately **6 months and 1 week (25 weeks total)**.

Associated Risks: Requires additional security measures to prevent leaks in the history log.

Approval: Pending

Additional Effort Estimate: 40-50 hours of development (mainly Backend Rust Dev and Senior Crypto & Security for history encryption).

Updated Team Configuration

The change request is incorporated without adding new roles. The team remains at 8 people. Responsibilities are adjusted to cover API history development, DELETE/EDIT operations and history encryption, Table 2. Team (8 people – 6 months + 1 week full-time).

#	Role	Level	Qty	Key Responsibilities (Updated)	Monthly Salary (MXN)	6.25-Month Total (MXN)
1	Sr. Crypto & Security	Senior	1	E2EE, anti-forensics, self-destruct, sandbox detection; encryption implementation for API history log.	68k – 75k	~445,625
2	Sr. Rust & Architecture	Senior	1	Binary obfuscation, Tor integration, anti-dumping; architecture design for DELETE/EDIT in User-API.	65k – 72k	~430,625
3	Frontend Developer	Mid	1	React, TS, D3.js, interactive dashboards; UI updates for API editing.	32k – 42k	~232,500
4	Backend Rust Dev	Mid	1	Data Listener, Network Analysis, Process Tracking; API history integration in Module 1 and DELETE/EDIT operations.	32k – 40k	~225,000
5	Integration / Mobile	Mid/Jr +	1	Tauri 2.0, IPC, Rust ↔ JS commands, bug fixing; mobile integration of audit history.	28k – 38k	~207,500

6	Full-Stack / Support	Jr/Mid	1	General FE/BE support, cross-cutting tasks; support in testing new API features.	25k – 35k	~188,750
7	Tester / Security Eng.	Senior	1	Pentesting, OWASP Mobile, E2EE validation; security validation for API history and auditing.	45k – 55k	~313,750
8	PM / Scrum Master	—	1	Coordination, backlog, agile ceremonies; change request management and timeline adjustments.	22k – 28k	~157,500

Table 2. Team (8 people – 6 months + 1 week full-time)

Personnel Cost Summary (Estimated Average):
2 Senior Developers (Crypto/Architecture): ≈ MXN 876,250
4 Mid/Junior Developers: ≈ MXN 853,750
1 Senior Security Tester: ≈ MXN 313,750
1 Part-Time Project Manager: ≈ MXN 157,500

Total Personnel Cost for 6.25 Months ≈ MXN 2,201,250 – 2,450,000 (Increase of ~MXN 98,250 – 100,000 due to the 1-week extension).

Updated Material Resources

No changes.

Updated Project Phases

Adds 1 week total, distributed across affected phases (Core Development, Security & Anti-Forensics, Intensive Testing). Timeline extended to 25 weeks Updated Table 3. Project Phases.

Phase	Weeks	% of Total Time	Main Activities (Updated)	Roles with Highest Effort	Important Notes
1. Initiation & Definition	1–2	~8%	Kick-off, detailed requirements, final architecture, security design, threat modeling; update of functional requirements for DELETE/EDIT and API history.	PM, Senior Crypto #1, Senior Rust	Critical phase — must not be skipped.
2. Crypto Research & Prototyping	3–5	~12%	Primitive validation, anti-forensics PoC; hashing prototype for history.	Senior Crypto #1, Senior Rust	Determines use of standard libraries.
3. Core Development (Rust)	4–13	~40%	Rust core implementation: entities, Data Listener, etc.; DELETE/EDIT and API history integration.	Senior Rust, Mid Rust	Longest phase — +1 week added.
4. Frontend Development	6–14	~36% (parallel)	Dashboard, timelines, charts, drag-and-drop; UI for API editing.	Mid Frontend, Junior Full-Stack	Starts after minimal backend.
5. Multiplatform Integration	10–16	~28% (parallel)	Rust ↔ React bridge, IPC, mobile builds; history integration.	Mid Integration, Mid Frontend	Mobile-specific integration.
6. Security & Anti-Forensics	8–19	~48% (cross-cutting)	E2EE implementation, obfuscation, sandbox detection, self-destruct; security for API history.	Senior Crypto #1, Senior Tester	Continuous effort — +1 partial week.
7. Intensive Testing + QA	14–22	~36%	Unit, integration, E2E (Appium), pentesting, anti-forensics validation; testing of DELETE/EDIT and history.	Senior Tester, all developers	+1 week added.

8. Polishing, Performance & Beta	19–23	~20%	Optimizations, bug fixing, real device testing, closed beta.	Entire team	Includes testing on 5 physical devices.
9. Closure & Delivery	23–25	~12%	Final documentation, delivery, training, final packaging; inclusion of API audit manual.	PM, Senior Rust, Tester, Mid Integration	Buffer for unforeseen issues.

Updated Table 4. Project Scale

Nombre	Descripción
Critique	Mandatory immediate implementation
High	High priority, next sprint / main phase
Medium	Desirable improvement, can be planned
Low	Optional / post-launch

Updated Table 4. Stage

General Guidelines Updated:

1. The current project stage (phases 1-9 of the updated 25-week timeline).
2. The scale of the change (additional hours, weeks of impact, affected modules, security risks, extra cost, and requester).

3-6-2026

Law mandatory

Scales defining

Homewkors, define the scales, guidelines

Foda, incluirlo

Actualizar el repositorio para esta semana

Carlos sole, becas de movilidad, biblioteca rampa arriba al fondo